Computer Engineering Senior Project Proposal
B. Eng. Computer Engineering
Academic Year 2568

Project Title: Smart Monitoring System for Personalized Gait Anomaly Detection in Elderly

Group Member

1. Tinnawut Noinarai 65070503447
2. Parunchai Kochseni 65070503456
3. Kanchai Lerdsrisakulrat 65070503475

Advisor: Dr. Piyanit Ua-areemitr

Signature:  _____ (Advisor)

(Dr. Piyanit Ua-areemitr)

Section 1

1.1 Project introduction & problem statement

In recent decades, the world has been facing a rapid demographic shift, with the number of people aged 60 and above expected to rise from 1 billion in 2020 to 2.1 billion by 2050 [1]. This aging population brings new healthcare challenges, as older adults are more likely to experience chronic conditions and geriatric syndromes. Among these, falls represent one of the most critical risks due to their severe physical, psychological, and economic consequences.

Globally, falls are a leading cause of injury, hospitalization, and loss of independence among older adults. For families and healthcare systems, they also create a substantial financial burden through medical treatment, rehabilitation, and long-term care. Despite extensive prevention campaigns, current approaches often remain reactive, intervening only after falls have already occurred. Gait anomalies such as increased variability and reduced cadence are strong predictors of falls in older adults. Evidence shows that these changes often reflect underlying disease rather than normal aging, with higher stride-to-stride variability and slower walking pace both independently linked to elevated fall risk [2–4]. This project addresses the need for proactive solutions by exploring wearable-sensor-based monitoring of walking patterns. By continuously analyzing gait data, it is possible to detect subtle deviations that may indicate an increased risk of falling. Using anomaly detection techniques, the system aims to provide early alerts that translate into actionable insights, enabling caregivers to initiate medical intervention and rehabilitation, which are crucial to preventing falls rather than responding after they occur.

1.2 Objectives

1. To develop a monitoring system that employs Machine Learning to accurately identify personalized gait anomalies by comparing each individual's walking patterns against their own baseline.
2. To design and implement a hardware prototype capable of collecting the sensor data required for model training, validation, and visualization.
3. To develop a mobile application that provides real-time monitoring and alerts, notifies caregivers of critical changes, and offers access to historical reports for tracking gait deterioration.

1.3 Scope

1.3.1 Solutions

This project aims to develop a Smart Monitoring System for Personalized Gait Anomaly Detection in Elderly. The system employs wearable sensors and machine learning models to derive real-time gait data, construct individual baselines, and notify caregivers of anomalies that may indicate increased fall risk before falls. As an alternative to mainstream threshold-based devices, the solution aims at early fall prevention as opposed to post-fall detection. Core functionalities include real-time notification to caregivers, history reporting, cumulative risk scoring, progress tracking and caregiver dashboard to aid timely intervention and ease the care burden.

1.3.2 Features

Wearable device for gait data collection : Prototype built on an embedded platform with IMU sensors to continuously capture gait signals (acceleration, angular velocity, orientation) and transmit them wirelessly. The device supports both battery for daily mobility and wired power supply for laboratory testing.

Walking detection model : Walk detection module isolates walking segments from continuous sensor data, ensuring only valid walking data are used for baseline training.

Anomaly detection model : Deep learning model learns normal gait cycles and flags deviations as early indicators of instability or fall risk.

Personalized baseline adaptation : System maintains and updates a user-specific gait baseline to minimize false positives and account for gradual gait changes over time.

Mobile application : Provides real-time monitoring, anomaly alerts, long-term trend visualization, and historical reports for caregivers and users.

1.3.3 Target Users And Operating Environment

The primary target users for this system are caregivers rather than elderly individuals themselves. Many elderly people may not recognise the need for such a device, believing their mobility and health remain normal, and therefore are unlikely to

purchase or use it on their own initiative. Caregivers, on the other hand, often experience significant concern when leaving elderly family members alone, especially in situations where a fall or gait abnormality could go unnoticed until it is too late. This concern is even greater for elderly individuals with cognitive impairments, who may require continuous and unobtrusive monitoring. By focusing on caregivers, the system addresses a motivated user group that actively seeks tools to ensure the safety and wellbeing of those under their care.

In addition to caregivers, a secondary target group includes elderly individuals who are relatively familiar with technology and capable of using smart devices independently. This group may be motivated to adopt such a system for self-monitoring, early detection of mobility decline, or maintaining independence without constant reliance on caregivers.

1.4 Project Plan

1.4.1. First Semester

1.4.1.1 Research

- Target research.
- Identify the objective of the project.
- Identify scope of work.
- Learn and research about Embedded, AI/ML, Software development tools that will be used.

1.4.1.2. Proposal

- Proposal report.
- Project Progress Presentation Preparation.

1.4.1.3. IoT and data communication

- Plan and Research pipeline of data from sensors to server.
- Develop wearable devices (retrieve sensor data).
- Model design for holding sensor and development board on target location.
- Design data communication from sensors to server.
- Setting up a cloud server for data communication.
- Test data sent from wearable devices to cloud servers.

1.4.1.4. Preparation for Frontend Development

- User Interface Design.
- Prototype Development.

1.4.1.5. Backend Development

- Backend Architecture Design.
- Backend Implementation.

1.4.1.6. Machine Learning - Walk Detection Model

- Clean and Segment Data.
- Prepare Data for a Model.
- Train and Test Model.

1.4.1.7. Set up baseline creation

- Preprocess and Window Data.
- Train model.
- Establish Error Threshold.

1.4.1.8. Project Progress

- Progress report.
- Progress Presentation Preparation.

1.4.2. Second Semester

1.4.2.1. Baseline creation and anomaly detection (Continue)

- Set up a live data pipeline.
- Reconstruct and Compute Error.
- Compare Threshold and Flag with Log and Review Anomalies.
- Finalize model for production.

1.4.2.2. Model Deployment and Integration

- Prepare for target runtime.
- Develop Inference Service.
- Integrate with Backend System.
- Deploy and Validate in Live Environment.

1.4.2.3. Frontend Developer

- User Interface Implementation.
- Backend API integration.
- Continuous Integration and Continuous Delivery.

1.4.2.4. Prototype Test

- User Acceptance Test.
- Feedback gathering.
- Document test result and issues occured.

1.4.2.5. Project Report

- Project report preparation.

1.5 Gantt chart

1.5.1 First Semester

Action
Tinnawut - Blue
All - Purple
Parunchai & Kanchai - magenta
Tinnawut & Kanchai - orange

Sub-tasks	August				Sep				Oct				Nov				Dec			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1. Research																				
1.1 Target research																				
1.2 Identify objective of project																				
1.3 Identify scope of work																				
1.5 Learn and research about Embeded, AI/ML, Software development tools																				
2. Proposal																				
2.1 Proposal report																				
2.2 Project Progress Presentation Preparation																				
3. Iot and data communication																				
3.1 Plan & Research pipeline of data from sensors to server																				
3.2 Develop wearable device (retreive sensor data)																				
3.3 Model design for holding sensor and development board on target location																				
3.4 Design data communication from sensors to server																				
3.5 Setting up cloud server for data comunication																				
3.6 Test data sending from wearable device to cloud server																				
4. Preparation for Frontend Development																				
4.1 User Interface Design																				
4.2 Prototype Development																				
5. Backend Development																				
5.1 Backend Architecture Design																				
5.1 Backend Implementation																				
6. ML - walk detection model																				
6.1 Clean & Segment Data																				
6.2 Prepare Data for Model																				
6.3 Train & Test Model																				
7. Set Up Baseline creation																				
7.1 Preprocess & Window Data																				
7.2Train model																				
7.3 Establish Error Threshold																				
8. Project Progress																				
8.1 Progress report																				
8.2 Progress Presentation Preparation																				

Figure 1.1: First Semester's Gantt Chart

1.5.2 Second Semester

Action
Tinnawut - Blue
All - Purple
Parunchai & Kanchai - magenta
Tinnawut & Kanchai - orange

Sub-tasks	January				February				March				April				May			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1. Baseline creation & anomaly detection (Continue)																				
1.1 Set Up Live Data Pipeline																				
1.2 Reconstruct & Compute Error																				
1.3 Compare to Threshold & Flag with Log & Review Anomalies																				
1.4 Finalize model for production																				
2. Model deployment & Integration																				
2.1 Prepare for target runtime																				
2.2 Develop Inference Service																				
2.3 Integrate with Backend Systems																				
2.4 Deploy and Validate in Live Environment																				
3.Frontend Development																				
3.1 User Interface Implementation																				
3.2 Backend API integration																				
3.3 Continuous Integration & Continuous Delivery																				
4. Prototype Test																				
4.1 User Acceptance Test																				
4.2 Feedback gathering																				
4.3 Document test result & issues occured																				
5. Project Report																				
5.1 Project report preparation																				

Figure 1.2: Second Semester's Gantt Chart

1.6 Deliverables

1.6.1. First Semester

- Proposal report.
- Proposal presentation.
- User interface mock up.
- Wearable device prototype.
- Sensor mounting model.

1.6.2. Second Semester

- Production-ready anomaly detection model.
- Operational inference service capable of processing incoming data and returning results.
- Mobile Application, fully implemented frontend integrated with backend APIs, displaying live and historical gait metrics and anomaly alerts.
- Gait Anomaly Detection Device.
- Final Project Report.

Section 2

2.1 Backgrounds

As described in Section 1.1, falls represent a major health challenge in older adults, and early detection of gait changes is a promising strategy for prevention. To support such monitoring, wearable sensing technologies have emerged as practical tools for capturing movement in daily life. In particular, Inertial Measurement Units (IMUs) are widely used because they are lightweight, non-invasive, and capable of continuously recording body motion through accelerometer and gyroscope signals.

From these raw signals, researchers have developed various gait indicators that provide clinically meaningful insights into walking stability and fall risk. In this project, five indicators are emphasized: Stride Time Variability, Cadence, Stance/Swing Ratio, Peak Angular Velocity, and Symmetry Index. These features are well established in gait research as they capture both temporal aspects (timing and rhythm) and dynamic aspects (force, symmetry, and balance) of walking, while remaining feasible to extract from wearable IMUs.

For example, Stride Time Variability (STV) quantifies how consistent the timing of steps is, with higher variability linked to instability and fall risk [2]. Cadence, or steps per minute, reflects overall rhythm and has been associated with mobility decline and frailty in older populations [4]. The Stance/Swing Ratio compares the proportion of time spent with the foot on the ground versus in motion, and deviations often signal compensatory balance strategies [5]. Peak Angular Velocity, derived from gyroscope measurements, reflects the strength and smoothness of leg movements, which typically decrease with age [6]. Finally, the Symmetry Index compares left and right step parameters to reveal imbalances often present in neurological or musculoskeletal conditions [8].

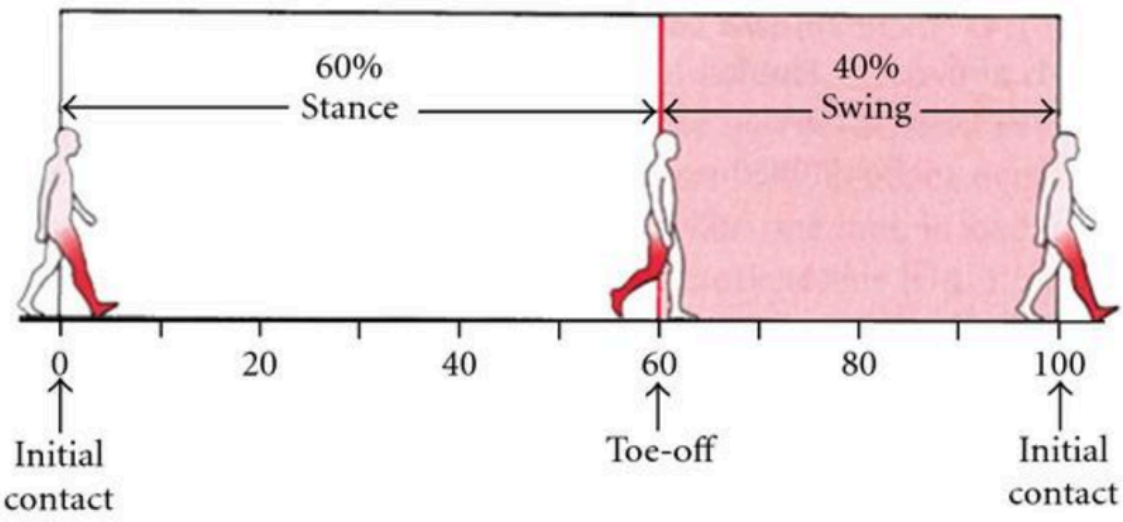


Figure 2.1: The gait cycle phases
https://www.researchgate.net/figure/The-gait-cycle-can-be-divided-into-the-stance-phase-weight-bearing-and-swing-phase_fig1_230763961

The theoretical foundation for these measures is the gait cycle, which is divided into stance and swing phases. As illustrated in Figure 2.1, a normal gait cycle consists of approximately 60% stance and 40% swing. These proportions provide the baseline for interpreting ratios and dynamics, such as the stance/swing ratio and angular velocity. Section 2.1.2 will describe in detail how each indicator is calculated from IMU data.

After receiving the sensor data, the following subsections will expand on additional concepts necessary for this study. Section 2.1.1 introduces the theory and calculation of gait indicators, followed by Section 2.1.2, which discusses visualization theory for mobile health applications. Section 2.1.3 presents the algorithmic framework, including the walk detection module (2.1.3.1) and the anomaly detection model (2.1.3.2). Section 2.1.4 then explains the Message Queuing Telemetry Transport (MQTT) protocol, while Section 2.1.5 outlines the Clean Architecture approach. Hardware considerations are described in Section 2.1.6, and sensor placement strategies in Section 2.1.7. Finally, Sections 2.1.8 and 2.1.9 provide an overview of the tools employed and related products, respectively.

2.1.1 Gait Indicator Theory and Calculation

In this work, all gait indicators are derived from IMU signals after basic preprocessing. The raw accelerometer data (ax, ay, az) and gyroscope data (gx, gy, gz) are first synchronized and filtered to remove noise. To reduce sensitivity to sensor orientation, the three accelerometer axes are combined into the resultant acceleration magnitude (RAM):

$$RAM(t) = \sqrt{ax(t)^2 + ay(t)^2 + az(t)^2} \quad (2.1)$$

This transformation preserves the periodic pattern of walking and makes peak detection more robust. The continuous signal is then segmented into overlapping windows (3 seconds with 50 % overlap). Only windows classified as “walking” by the activity recognition module are processed further.

Stride Time Variability (STV). When a heel strikes the ground, RAM(t) produces a distinct peak due to the impact. The interval between two consecutive peaks from the same foot defines the stride time. Across a walking session, the mean stride time (μ) and standard deviation (σ) are computed, and STV is defined as:

$$STV = (\sigma\ stride\ time \div \mu\ stride\ time) \times 100\% \quad (2.2)$$

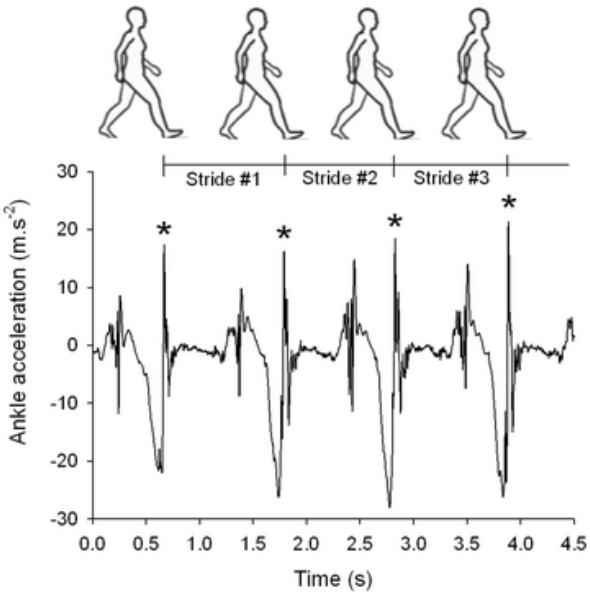


Figure 2.2: Stride time visual
https://www.researchgate.net/figure/Stride-duration-determination-Peak-detection-method-Heel-strikes-correspond-to-each_fig1_313900243

Figure 2.2, shows heel-strike and toe-off events in a gait cycle. If stride intervals are highly similar, STV is low; if they fluctuate more, STV is high, which has been linked to unstable gait in older adults [2,3].

Cadence reflects the number of steps per minute. Since each step produces a peak in RAM(t), cadence can be calculated directly as:

$$Cadence \; = \; (\, Number\ of\ steps\ in\ window \,/\, Window\ duration\ (minutes\,)) \qquad (2.3)$$

Alternatively, the dominant frequency (f_peak) in RAM(t) within the walking band (0.8–2.5 Hz) can be extracted and multiplied by 60 [9].

Stance-to-Swing Ratio. The stance phase occurs when the foot is on the ground, and the swing phase when it is in the air. Gyroscope signals are well-suited for distinguishing these phases. The combined magnitude is:

$$GyroMag(t) \; = \; \sqrt{gx(t)^2 \; + \; gy(t)^2 \; + \; gz(t)^2} \qquad (2.4)$$

During stance, leg rotation is minimal, so GyroMag(t) remains low. During swing, it increases sharply. By summing the durations of low-rotation and high-rotation segments, the stance/swing ratio is:

$$Stance/Swing\ Ratio \; = \; Total_stance_time \,/\, Total_swing_time \qquad (2.5)$$

In healthy adults, this ratio is typically close to 1.5 (≈ 60 % stance, 40 % swing) [10]. Figure 2.1 illustrates the gait cycle, showing stance and swing contributions.

Peak Angular Velocity. During the swing phase, leg rotation accelerates and reaches a maximum just before heel strike. By extracting the peak of GyroMag(t) in each swing and averaging over the session, peak angular velocity is obtained. Lower values may indicate muscle weakness or restricted mobility [6].

Symmetry Index (SI). Because steps alternate between left and right legs, odd and even peaks in RAM(t) can be separated. The average step times for left (X_L) and right (X_R) sides are compared using:

$$SI = (|X_L - X_R| / (0.5 \times (X_L + X_R))) \times 100\% \quad (2.6)$$

An SI near zero indicates symmetry, while higher values reflect imbalance. This measure is commonly applied in musculoskeletal and neurological studies [7,11].

By systematically linking RAM(t) and GyroMag(t) to these indicators, the system transforms raw IMU signals into clinically meaningful features. This enables consistent monitoring of gait changes and supports early detection of deterioration or improvement in walking stability.

2.1.2 Visualization Theory for Mobile Health Applications

Data Visualization in Health Monitoring

Visual representation of sensor-derived data plays a critical role in health monitoring systems. Compared to raw numerical outputs, graphs and charts enable users and caregivers to quickly recognize patterns, trends, and anomalies. Edward Tufte's principles of data visualization emphasize clarity, minimal distortion, and the importance of comparisons to ensure that data communicates meaningful insights rather than confusion [12]. In the context of mobile health (mHealth), clear visualization is especially important for older adults, who may have limited digital literacy. Simple and intuitive displays, such as line graphs of stride variability over multiple weeks, help highlight gradual deterioration or improvement in gait quality without requiring technical interpretation. By translating complex IMU-derived metrics into visually interpretable formats, users and caregivers can better monitor progress and detect early warning signs.

Risk Communication and Scoring Models

For health-related applications, it is not enough to display raw measurements; data must also be translated into actionable insights. Risk communication theory suggests that complex information should be simplified into formats that users can immediately understand and act upon. One common approach is the use of scoring models or traffic-light systems (green/yellow/red) that summarize risk levels. The Fogg Behavior Model argues that simplicity and clear feedback improve adherence to behavioral recommendations [14]. Similarly, fall-prevention dashboards promoted by health organizations such as the WHO highlight that risk feedback should be concise and immediately actionable [1]. In practice, anomaly detection outputs can be transformed into risk categories such as "low," "medium," or "high" fall risk, helping caregivers prioritize interventions and motivating users to adjust their activity levels when necessary.

Time-Series Visualization

Fall risk is not static but evolves over time, making longitudinal visualization essential. Time-series plots allow both users and caregivers to observe changes across days, weeks, or months, providing context that single measurements cannot. According to Shneiderman's Visual Information-Seeking Mantra, visualization should allow an overview of long-term gait trends, while enabling focused analysis of specific periods [13]. For example, a weekly trendline of cadence may reveal whether the user is gradually slowing down, while a rolling average of stride time variability (STV) can smooth short-term fluctuations to highlight meaningful changes. Threshold-based highlighting, such as marking STV values above 6% in red, provides additional visual cues to detect when indicators cross clinically significant boundaries.

Personalization and Baseline Comparison

Population-based thresholds are useful for screening, but gait is highly individual. Personalized feedback therefore provides more meaningful insights. Studies in personalized medicine show that comparing a user's current measurements to their own historical baseline (N-of-1 trajectories) is more effective for detecting subtle changes that may indicate early decline [15]. For example, displaying "Your average STV last month: 4.2%, Today: 5.1% (+0.9%)" highlights deviations that might be overlooked if only compared to population norms. This approach also fosters user trust by showing that the system adapts to their unique walking style, rather than applying generic benchmarks. Such personalization is especially valuable in long-term monitoring, where gradual changes accumulate into meaningful trends.

User-Centered Design for Caregivers

While older adults are the primary subjects of gait monitoring, the end users of the system often include caregivers, clinicians, or family members who supervise health and safety. For these users, visualization must prioritize decision support by providing actionable insights and reducing the time needed to interpret raw data. According to human-centered design principles (ISO 9241-210), systems should be tailored to the needs of their target audience, emphasizing efficiency, clarity, and context-awareness [16]. Caregivers benefit from dashboards that summarize key indicators across individuals, highlight anomalies automatically, and allow prioritization of high-risk cases. For example, a monitoring interface may display a simple traffic-light alert system to flag which users show abnormal stride variability or gait asymmetry, while still allowing access to detailed graphs when deeper analysis is required. Research on interactive dashboards for informal caregivers confirms that timely and clear alerts, combined with intuitive visual summaries, can reduce caregiver burden and improve intervention rates [17]. In this way, the visualization layer functions not only as a reporting tool but also as a support system for proactive fall prevention and long-term health management.

2.1.3 Algorithm

The proposed system relies on two main algorithmic components: a walk detection model and an anomaly detection model. These modules work together to ensure that only relevant gait data is analyzed and that abnormal walking patterns can be identified at an early stage. The walk detection model first isolates walking activities from other daily movements such as sitting, standing, or lying down, so that subsequent processing is restricted to gait-relevant segments. The anomaly detection model then evaluates these walking segments to identify unusual patterns that may signal increased fall risk. Together, these algorithms create a robust pipeline that links raw sensor data to clinically meaningful insights.

2.1.3.1 Walk Detection Module

The walk detection module is the first stage of the system, ensuring that gait analysis is applied only to meaningful walking segments. Metrics such as stride time variability and stance-to-swing ratio cannot be reliably computed when the subject is sitting, standing, or performing unrelated activities. By filtering out non-walking periods, this module reduces false alarms and increases the accuracy of subsequent gait analysis.

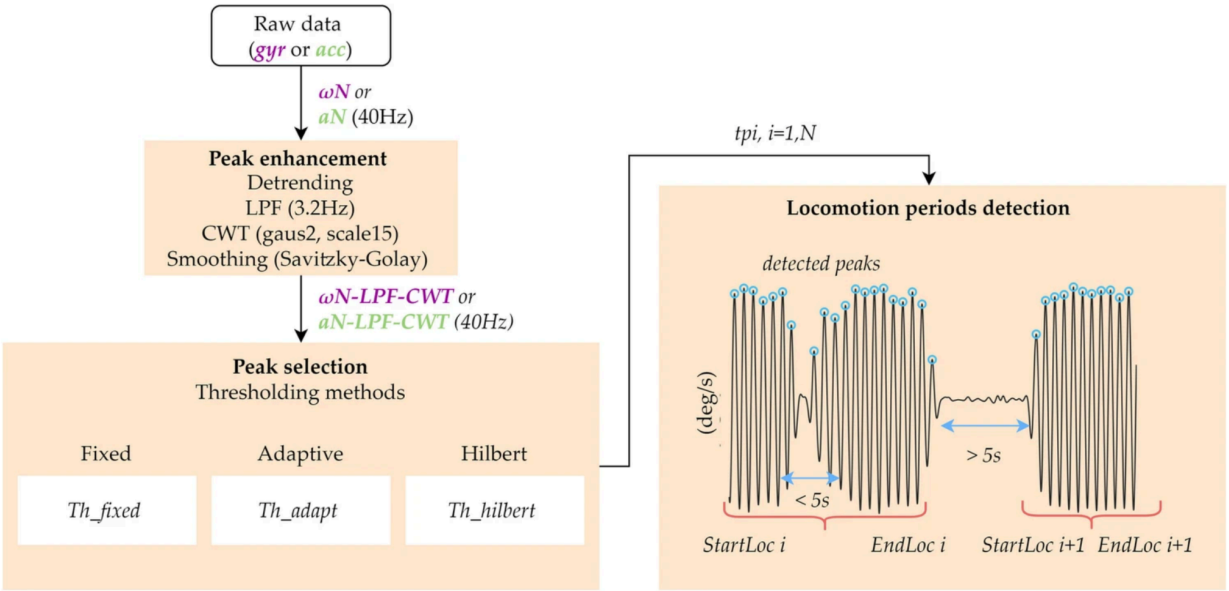


Figure 2.3: Flowchart of the walking detection algorithm
https://link.springer.com/article/10.1007/s11517-023-02826-x/figures/2

The overall workflow for preparing raw IMU data prior to classification is illustrated in Figure 2.3. This diagram outlines the transformation from raw three-axis accelerometer and gyroscope signals into a compact set of discriminative features suitable for walking detection. It shows the key stages which are filtering, segmentation, and feature extraction that ensure the data is both robust and computationally efficient for real-time processing.

In this work, IMU data is sampled at 50–100 Hz and first passed through a low-pass filter with a 4–6 Hz cut-off to remove high-frequency noise. The gravity component is then removed from accelerometer signals using high-pass filtering or sensor fusion. The continuous IMU stream is segmented into fixed-length windows of three seconds with 50 % overlap, ensuring that transient gait events are captured without gaps.

From each window, features are extracted in both time and frequency domains. Time-domain features include mean, variance, standard deviation, signal magnitude area (SMA), zero-crossing rate, and peak count. Frequency-domain features include the dominant frequency obtained via FFT, spectral energy within the 0.8–2.5 Hz band corresponding to typical adult walking cadence [18], and spectral entropy. Cross-axis correlations between accelerometer and gyroscope axes are also computed to capture inter-axis motion relationships.

Following the approach validated by Kim et al. [19] and Springer et al. [20], a window is classified as “walking” if two conditions are met:

(1) the dominant frequency of the resultant acceleration magnitude (RAM) lies within the walking cadence band of 0.8–2.5 Hz, and

(2) the variance of the RAM exceeds a minimum threshold, distinguishing walking from static or low-movement activities.

If both conditions are satisfied, the window is labelled as walking; otherwise, it is discarded from further gait analysis.

Evidence from Kim et al. [19] shows that such a frequency and threshold approach achieved over 95 % accuracy for walking detection in healthy adults using smartphone IMU data. Springer et al. [20] validated a similar method in real-life settings with a single foot-worn sensor, achieving over 90 % accuracy even under variable walking speeds and terrains. These results indicate that, when thresholds are tuned to the target population and sensor placement, accuracy rarely drops below 85–90 % in realistic conditions.

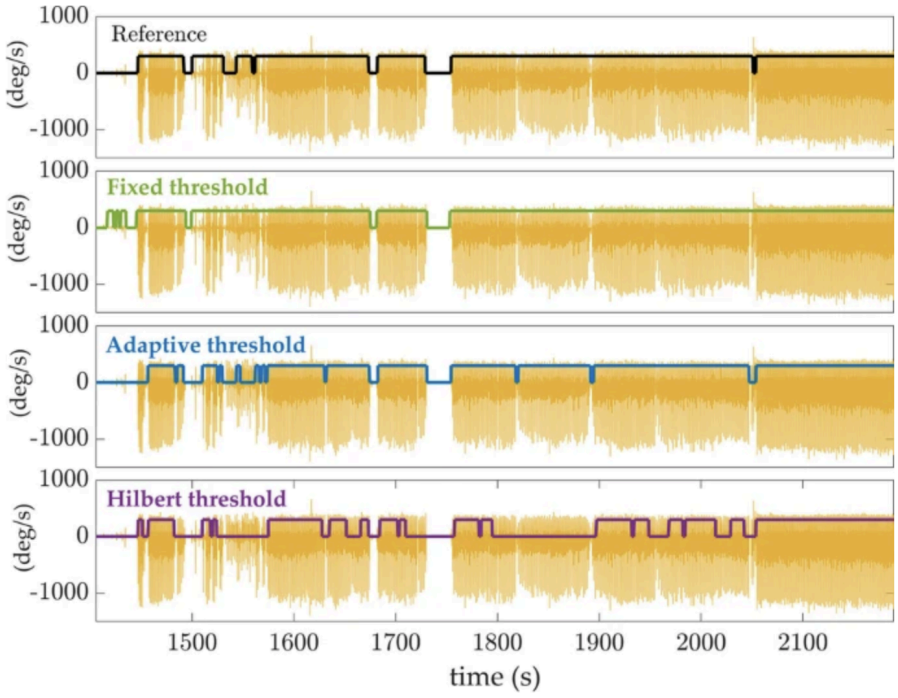


Figure 2.4: Walking detection using the different thresholding methods on same subject
https://link.springer.com/article/10.1007/s11517-023-02826-x/figures/5

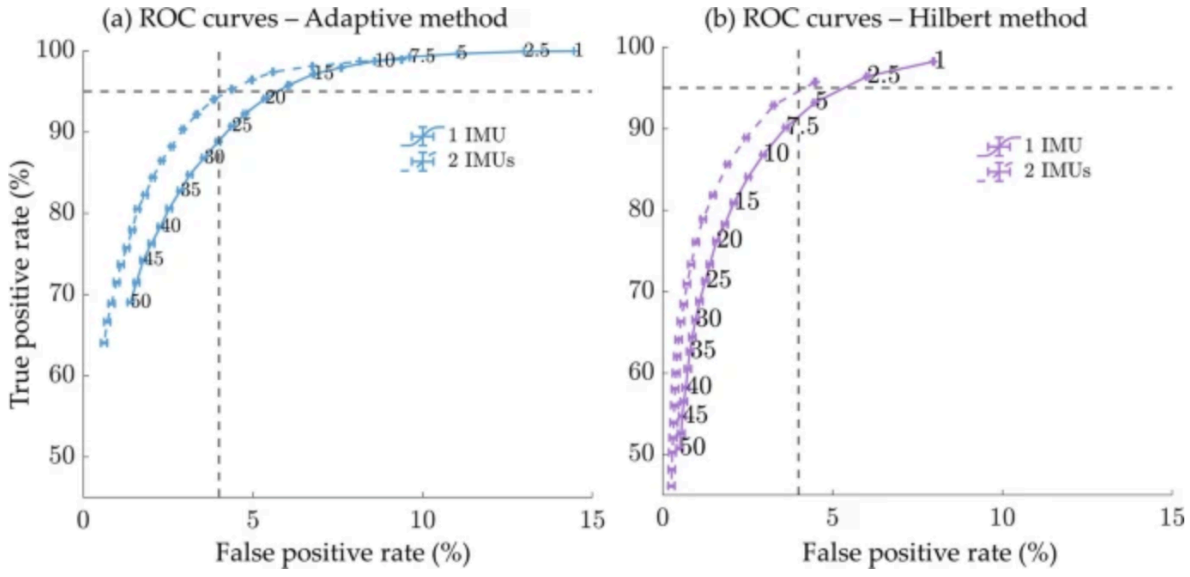


Figure 2.5: ROC curves for performance evaluation as a function of percentile values from 1 to 50% when the angular velocity norm ωN is used as input.
https://link.springer.com/article/10.1007/s11517-023-02826-x/figures/6

As illustrated in Figure 2.4, the fixed, adaptive, and Hilbert threshold methods produce visibly different detection patterns when applied to the same subject's angular velocity data. While the adaptive and Hilbert methods appear to align more closely with the reference classification in this example, a definitive conclusion on the best-performing method requires aggregated results across all subjects, as presented in the subsequent ROC analysis (Figure 2.5) [20].

2.1.3.2 Anomaly Detection Model

After walking segments are identified, they are passed into the anomaly detection model, which is based on a Long Short-Term Memory Autoencoder (LSTM-AE). The autoencoder framework is well suited for this problem because it compresses input data into a lower-dimensional representation and then reconstructs it. If the model has been trained only on normal gait sequences, it will reconstruct these sequences with low error. However, when abnormal or unusual gait patterns are input, the reconstruction error will be much higher, providing a natural way to detect anomalies [21].

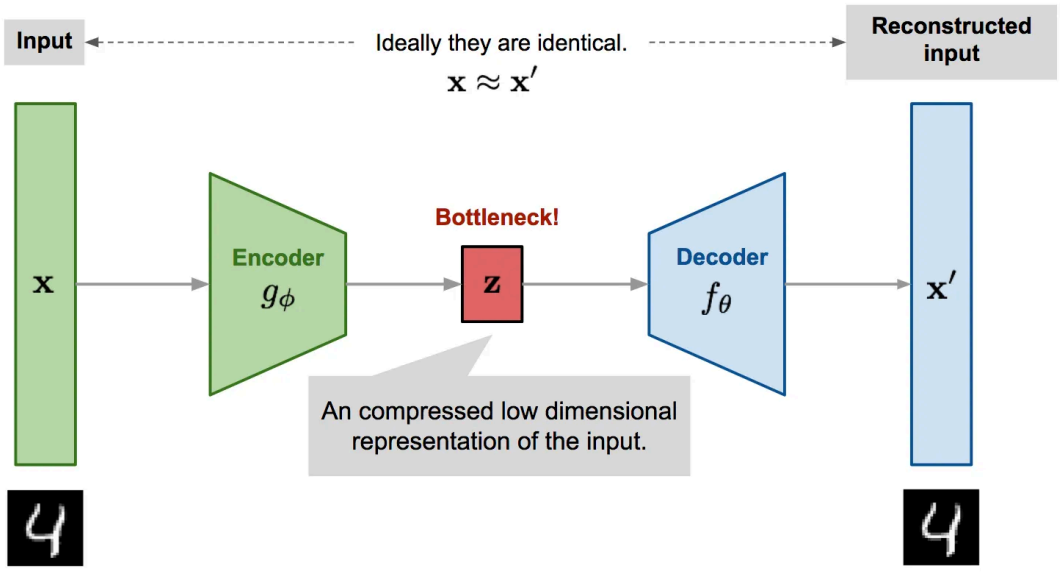


Figure 2.6: LSTM Autoencoder structure
https://ml-lectures.org/docs/unsupervised_learning/Anomaly_Detection_RNN_AE_VAE.html

Figure 2.6 illustrates the general structure of an autoencoder. The input sequence (x) is processed by the encoder, which uses LSTM units to capture temporal dependencies in the gait data. The

compressed latent representation, often referred to as the bottleneck (z), acts as a compact summary of the input sequence. The decoder then attempts to reconstruct the original sequence (x′) from z. Ideally, for normal walking patterns, the reconstructed signal (x′) closely matches the input (x). In contrast, abnormal gait sequences cannot be reconstructed accurately, leading to a higher reconstruction error.

The reconstruction error is typically measured using Mean Squared Error (MSE) between the input and reconstructed sequence:

$$E = (1/n)\,\Sigma\,(xi - x'i)^2 \qquad (2.7)$$

where xi is the original gait feature at time step i, and x′i is its reconstruction. During deployment, an error threshold is defined. If the reconstruction error of a walking window exceeds this threshold, the segment is flagged as an anomaly, indicating a possible gait irregularity.

The use of LSTM units in both encoder and decoder provides two key advantages. First, LSTMs are specifically designed to handle sequential data and long-term dependencies, which are inherent in gait cycles. Second, the recurrent structure allows the model to capture subtle timing irregularities, such as stride-to-stride variability, that simpler feedforward models may miss. This makes LSTM-AE particularly effective for gait-based fall risk assessment.

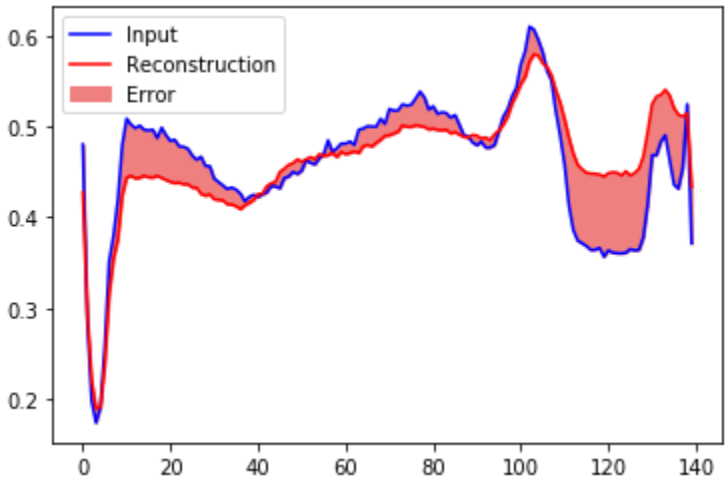


Figure 2.7: Reconstruction error plot example
https://ml-lectures.org/docs/unsupervised_learning/Anomaly_Detection_RNN_AE_VAE.html

Figure 2.7 shows an example of anomaly detection using reconstruction error. The blue line represents the input gait signal, while the red line represents the reconstructed output from the

autoencoder. The shaded region highlights the error between the two signals. For normal walking sequences, the reconstruction closely follows the input with small error margins. However, when gait becomes irregular, the error increases significantly, making it possible to identify anomalies through a defined threshold.

2.1.4 Message Queuing Telemetry Transport (MQTT) Protocol

Message Queuing Telemetry Transport (MQTT) is a lightweight publish–subscribe messaging protocol widely used in Internet of Things (IoT) applications. It was originally developed by IBM in 1999 to enable reliable communication over constrained networks and devices with limited resources. MQTT operates on top of the TCP/IP stack and follows a broker-based architecture, where clients communicate indirectly through a central broker.

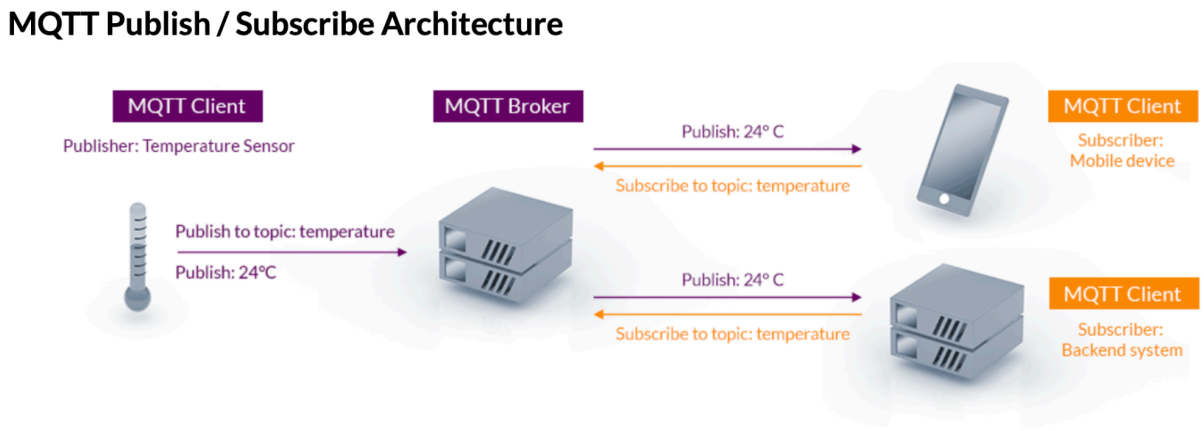


Figure 2.8: MQTT Publish/Subscribe Architecture https://mqtt.org/

Figure 2.8 The protocol defines two main roles: publishers, which send messages on specific topics, and subscribers, which receive messages by subscribing to those topics. The broker manages message distribution, ensuring decoupling between data producers and consumers. MQTT also provides Quality of Service (QoS) levels—*at most once (0)*, *at least once (1)*, and *exactly once (2)*—to balance performance and reliability depending on the application's requirements.

Due to its low bandwidth overhead, asynchronous communication model, and scalability, MQTT has become one of the most widely adopted standards for IoT systems, particularly in scenarios such as sensor data collection, remote monitoring, and real-time event-driven applications [22].

2.1.5 The Clean Architecture

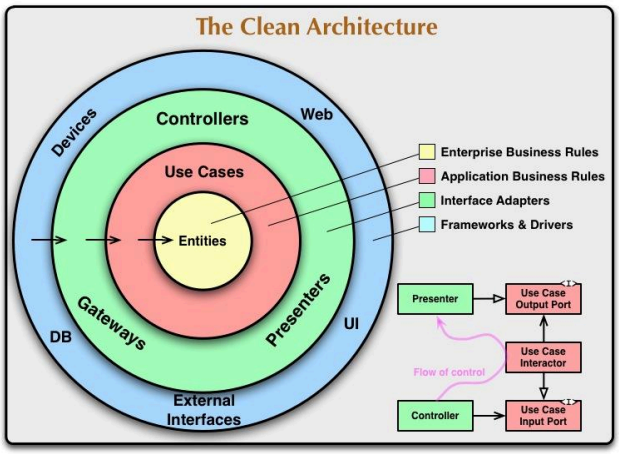


Figure 2.9: The Clean Architecture
https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.htm

Figure 2.9 shows that at the center of the architecture are Entities, which encapsulate enterprise-level business rules and remain the most stable components of the system. Surrounding them are Use Cases, which contain application-specific business logic and orchestrate interactions between entities. The next layer, Interface Adapters, translates data between the internal structures and external mechanisms such as databases, web frameworks, or user interfaces. Finally, the outermost layer, Frameworks and Drivers, includes tools and external technologies that the system depends on but should not affect the inner business rules [22].

By following this layered design, Clean Architecture produces systems that are independent of frameworks, testable, and adaptable to UI or database changes. This flexibility supports long-term maintainability and scalability, making it a widely adopted approach in modern software engineering.

2.1.6. Hardware

2.1.6.1 Microcontroller Unit (MCU)

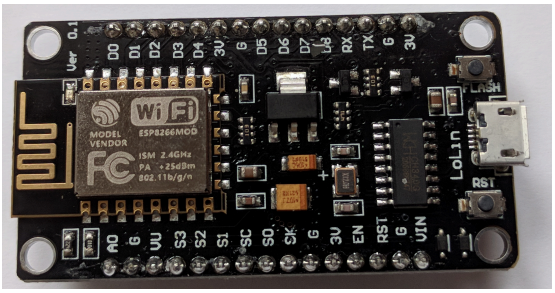


Figure 2.11: The picture shown ESP8266 [30]
https://annefou.github.io/IoT_introduction/02-ESP8266/index.html

The ESP8266 (Figure 2.11), developed by Espressif Systems, is a low-cost microcontroller with integrated Wi-Fi functionality that has become widely popular in Internet of Things (IoT) applications. It provides sufficient processing capability, built-in TCP/IP networking, and low power consumption, making it highly suitable for portable and wearable systems requiring wireless connectivity. Although it offers fewer peripherals and lower performance compared to newer models like the ESP32, the ESP8266 remains an efficient and cost-effective choice for projects emphasizing real-time data transmission and energy efficiency. In gait monitoring applications, the ESP8266 can act as the central unit to collect motion data from IMU sensors and transmit it to mobile or cloud platforms for further analysis [30].

2.1.6.2 Inertial Measurement Unit (IMU)

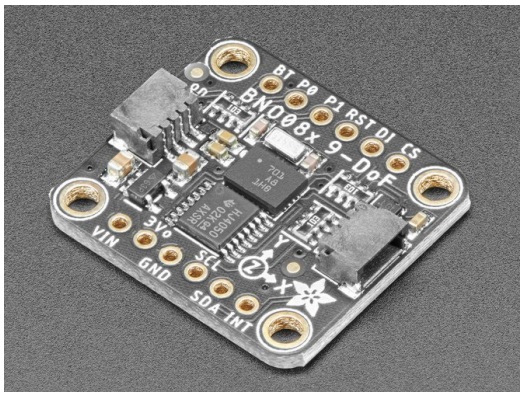


Figure 2.12: The picture shown BNO085 (IMU)
https://learn.adafruit.com/assets/95533

The BNO085(Figure 2.12) is a 9-degree-of-freedom (9-DoF) Inertial Measurement Unit (IMU) that integrates an accelerometer, gyroscope, and magnetometer, along with an embedded sensor fusion processor. This combination enables the device to provide accurate motion and orientation data, including linear acceleration, angular velocity, and absolute orientation. Unlike raw IMU sensors that require complex calibration and filtering, the BNO085 incorporates advanced sensor fusion algorithms directly on-chip, significantly reducing computational overhead on the host microcontroller. Its compact size, low power consumption, and ability to deliver reliable real-time data make it highly suitable for wearable applications, robotics, and motion tracking systems. In gait analysis, the BNO085 can capture precise kinematic parameters essential for detecting abnormalities and monitoring rehabilitation progress, while minimizing errors caused by sensor drift or environmental disturbances [31].

2.1.7 Sensor Placement in Gait Analysis

2.1.7.1 Influence of Sensor Placement on Gait Measurement

Gait analysis is an important tool for evaluating human movement and is commonly used in medicine, rehabilitation, and biomechanics research. Accurate assessment of gait can help identify movement problems and track patient progress over time.

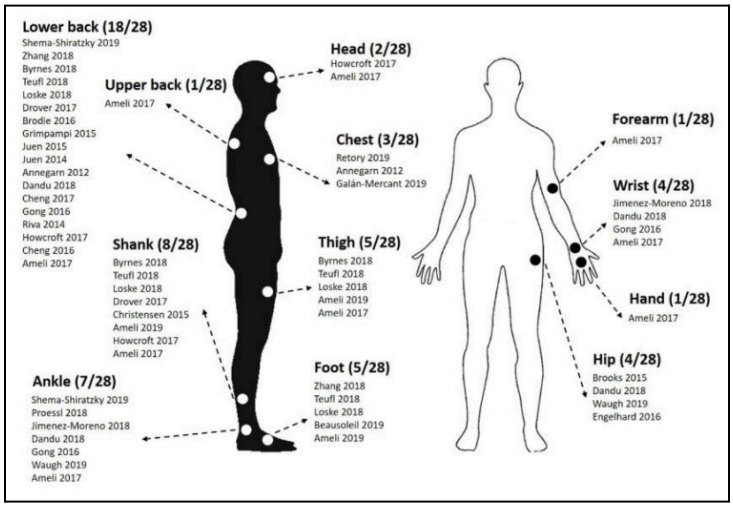


Figure 2.13: Sensor placement reported in the reviewed papers

https://pmc.ncbi.nlm.nih.gov/articles/PMC7249076/figure/sensors-20-02660-f002/

Recently, wearable sensors, such as accelerometers and gyroscopes, have been widely used because they can collect movement data in real time. The placement of these sensors on the body

is a key factor that affects the accuracy and reliability of the data. Studies have shown that placing sensors at different body parts such as the pelvis, thigh, ankle, and foot can influence the measurement of gait parameters, including walking speed, step length, and balance [33].

Choosing the best sensor locations requires consideration of several factors, including sensor stability during movement, the ability to collect relevant data, and user comfort. Proper sensor placement can improve the accuracy of gait analysis and support effective assessment and rehabilitation.

Therefore, understanding the optimal placement of wearable sensors for gait analysis is important for developing reliable monitoring systems that can be used in different clinical and research settings.

2.1.7.2 Optimal Sensor Locations for Accurate Gait Analysis

Since the placement of sensors affects the accuracy of measurements in gait analysis, it is important to determine the optimal sensor location to achieve the most precise results.

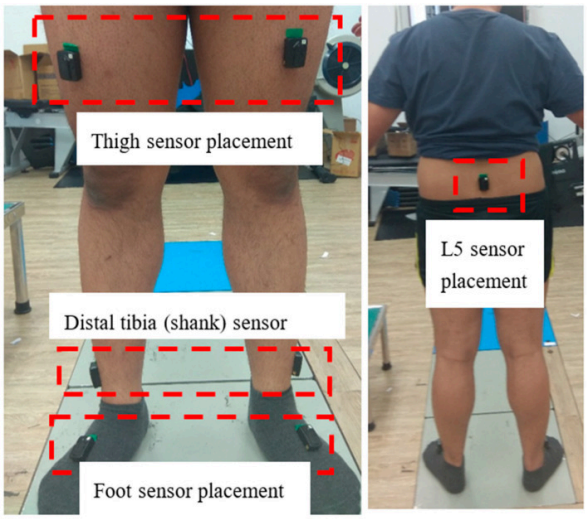


Figure 2.14: Illustration of the placement of the seven sensors on the subject

https://pmc.ncbi.nlm.nih.gov/articles/PMC6210399/figure/sensors-18-03397-f001/

In this study (Figure 2.14), seven IMU sensors were placed at different body locations, including the lower back (L5), both sides of the thigh, distal tibia (shank), and foot. Temporal parameters of gait were collected using these sensors, and various sensor configurations and features were analyzed to identify the most effective placement for classifying gait patterns.

The results showed that the sensor placed on the shank achieved the highest accuracy compared to other locations. Using a Decision Tree classifier, the shank sensor reached 89.13% testing accuracy in distinguishing between healthy individuals, stroke patients, and patients with other neurological disorders. Combining time-domain and gait temporal features as model input further improved classification performance.

These findings indicate that wearable IMU sensors can effectively differentiate gait patterns in neurological patients, and that placing the sensor on the shank provides the most reliable results for gait analysis and classification [32].

2.1.8 Development Tools

2.1.8.1 Software Frameworks

In modern software development, frameworks act as guiding blueprints that simplify how applications are built and maintained. Instead of starting every project from scratch, developers can rely on frameworks that provide ready-made structures, reusable components, and best practices. This not only speeds up the development process but also helps ensure that applications remain consistent, scalable, and easier to manage over time. In this section, three widely adopted frameworks React Native, Gorm, and Fiber are introduced to illustrate how different tools support different layers of application development, from mobile interfaces to database interaction and backend services.

React Native

React Native is an open-source framework developed by Facebook that enables developers to build mobile applications for both iOS and Android using a single JavaScript codebase. It allows the creation of native-like user interfaces by rendering components through native APIs rather than web views, ensuring high performance and responsiveness. React Native supports hot-reloading, a modular architecture, and a large ecosystem of reusable libraries, making it an efficient choice for cross-platform mobile development. Its flexibility and cost-effectiveness have established it as a leading framework for modern mobile applications [24].

Gorm

Gorm is a widely used Object-Relational Mapping (ORM) library for the Go programming language. It provides a simplified abstraction over database interactions, allowing developers to manipulate data using Go structures instead of raw SQL. Gorm supports essential features such as migrations, associations, transactions, and hooks, enabling developers to implement complex

database logic with minimal effort. Its extensibility and compatibility with major relational databases (e.g., MySQL, PostgreSQL, SQLite, SQL Server) make it a popular choice for Go-based backend systems that require reliability and maintainability [25].

Fiber

Fiber is a fast, lightweight, and expressive web framework built on top of Go's Fasthttp package. Inspired by Express.js, Fiber provides a familiar routing API while optimizing for performance and low memory usage. It offers features such as middleware support, template rendering, static file serving, and WebSocket integration, enabling developers to build scalable web services and APIs efficiently. Fiber's minimal footprint and high throughput performance make it particularly suitable for microservices architectures and real-time applications where speed is critical [26].

2.1.8.2 Database and Data Processing Tools

This section aims to provide a deeper exploration of databases and data processing tools, explaining their core characteristics, functionalities, and significance within the technological environment. Readers can develop a heightened awareness of their pivotal role in the management and utilization of data across a wide array of applications.

Postgres

PostgreSQL is a powerful open-source object-relational database management system (ORDBMS) that extends the SQL standard with a wide range of advanced features. Originating from the POSTGRES project at the University of California, Berkeley in 1986, it has undergone more than 35 years of continuous development, establishing a reputation for reliability, data integrity, and extensibility. PostgreSQL supports diverse data types, including structured (arrays, ranges), semi-structured (JSON, XML, hstore), and geospatial formats through extensions such as PostGIS. It is fully ACID-compliant, ensuring transactional consistency, and implements Multi-Version Concurrency Control (MVCC) to manage concurrent operations efficiently. Additional features such as indexing strategies, partitioning, replication, and point-in-time recovery make it suitable for both small-scale applications and large-scale enterprise systems. Its extensibility is a core strength, allowing developers to define custom data types, operators, and functions, or even integrate procedural languages like Python, JavaScript, or Rust directly into the database. Combined with strong community support and wide adoption across industries, PostgreSQL has become one of the most trusted relational databases for managing complex and large-scale workloads [27].

Redis

Redis (Remote Dictionary Server) is an open-source, in-memory data structure store that can be used as a caching mechanism, message broker, and key-value database. It is designed for high-performance applications that require fast data retrieval and manipulation. Redis stores data entirely in memory, allowing for extremely fast read and write operations. It operates as a key-value store, where data is associated with a unique key, enabling efficient data retrieval. Redis is categorized as a NoSQL database due to its non-relational data storage model. It supports various data structures, including strings, lists, sets, sorted sets, hashes, bitmaps, and geospatial indexes, providing versatility for different use cases [28].

2.1.8.3 IoT and Messaging Platforms

This section introduces IoT and messaging platforms that provide device connectivity, data communication, and integration with cloud infrastructures. Such platforms enable reliable, scalable, and secure data exchange in distributed systems, which is essential in IoT and real-time applications.

AWS IoT Core
AWS IoT Core is a managed cloud service by Amazon Web Services that allows IoT devices to securely connect and interact with cloud applications and other devices. It supports multiple communication protocols, primarily MQTT, alongside HTTPS and WebSockets, making it suitable for lightweight and constrained devices. AWS IoT Core automatically scales to handle billions of messages, offering features such as device authentication, fine-grained access control, and integration with other AWS services (e.g., AWS Lambda, DynamoDB, or S3) for storage, processing, and analytics. Its managed infrastructure significantly reduces the complexity of building IoT ecosystems, enabling developers to focus on application logic rather than connectivity management [29].

2.1.8.4 Tools for Gait Analysis

Data Handling and Pre-processing

NumPy NumPy is a fundamental Python library for numerical computing, widely used in scientific and engineering applications. It provides support for multi-dimensional arrays and a wide range of mathematical functions for linear algebra, signal processing, and statistical operations. In this project, NumPy is used for handling raw IMU sensor data efficiently, enabling fast computations such as filtering, normalization, and feature extraction. Its array-based

operations are highly optimized, making it well-suited for processing the large amounts of time-series data generated during gait analysis [34].

Pandas Pandas is an open-source Python library for data manipulation and analysis, built on top of NumPy. It provides powerful data structures such as DataFrames and Series, which allow for efficient handling of tabular and time-series data. In this project, Pandas is used to organise, clean, and transform IMU sensor data, enabling operations such as resampling, merging datasets, and preparing features for both walk detection and anomaly detection models [35].

SciPy SciPy is a scientific computing library built on top of NumPy that provides advanced functionality for mathematics, optimization, and signal processing. It includes modules for filtering, Fourier transforms, and statistical analysis, all of which are critical for preprocessing IMU signals. In this project, SciPy is particularly used for filtering raw accelerometer and gyroscope data, extracting frequency-domain features, and performing interpolation to prepare clean signals for detection and classification [36].

Machine Learning and Modelling

Scikit-Learn Scikit-Learn is a widely used Python library for machine learning and statistical modeling. It provides tools for classification, regression, clustering, and dimensionality reduction, along with utilities for model evaluation. In this project, Scikit-Learn is employed for feature-based classification in the walk detection stage, enabling the differentiation of walking from non-walking activities using algorithms such as decision trees or k-nearest neighbors. Its lightweight nature and ease of use make it practical for rapid prototyping and integration into wearable systems [37].

TensorFlow TensorFlow is an open-source machine learning framework developed by Google, widely used for building, training, and deploying deep learning models. It supports both high-level APIs (such as Keras) for rapid prototyping and low-level operations for fine-grained control over model architecture. In this project, TensorFlow is used to implement and train the anomaly detection models, including recurrent architectures such as LSTM, enabling efficient handling of sequential gait data. Its GPU acceleration and model-saving capabilities facilitate both experimentation and later integration into deployment pipelines [38].

Visualisation

Matplotlib Matplotlib is a comprehensive Python plotting library for creating static, animated, and interactive visualisations. It is used in this project to generate detailed plots of sensor signals,

model training curves, and detection outputs. By providing precise control over figure elements such as axes, labels, and annotations, Matplotlib ensures that visual results are publication-quality and suitable for inclusion in technical reports and academic manuscripts [39].

Seaborn Seaborn is a statistical data visualisation library built on top of Matplotlib, offering a high-level interface for creating attractive and informative graphics. It simplifies the creation of complex plots such as heatmaps, violin plots, and distribution plots, which are used in this project to explore feature distributions, visualise correlations, and summarise model performance metrics. Seaborn's integration with Pandas data structures allows for efficient visual analysis of large sensor datasets [40].

Development Environment

Google Colab Google Colab is a free, cloud-based Jupyter Notebook environment that allows users to write and execute Python code through a web browser without local setup. It provides free access to computing resources, including GPUs, which is valuable for training and testing machine learning models. In this project, Google Colab is used for developing and testing both walk detection and anomaly detection algorithms, enabling collaborative code sharing and rapid experimentation without the need for dedicated hardware [41].

Model Deployment

FastAPI FastAPI is an open-source Python web framework designed for building high-performance APIs with minimal code. It supports asynchronous request handling, automatic data validation, and interactive API documentation. In this project, FastAPI is considered for the prototype deployment stage to wrap the trained anomaly detection model into a RESTful service, enabling quick, no-cost integration with the backend for testing and evaluation [42].

2.1.9 Existing Products

Currently, there are multiple devices and products that can detect and analyze gait abnormalities as well as provide fall detection and alerts. Examples include consumer-oriented devices such as smartwatches and medical-grade solutions like IMU sensors. However, most existing products only provide data for users to interpret on their own, without offering a structured system for rehabilitation tracking after accidents. This often leads to confusion or misinterpretation.

Figure 2.10 compares existing products in terms of their main features and specifications, together with our proposed product.

Feature/ Device	RunScribe Plus	Apple Watch Series 10	Samsung Galaxy Watch 8	Shimmer3 IMU Unit	Whoop 5.0 MG LIFE	Our Product
Price	฿18,000–22,000	฿14,900–15,900	฿11,900–13,900	฿30,000+	฿19,000–21,000 + annual subscription fee	฿8,000-10,000
Sensor Placement	Mounted on the shoe	Wrist	Wrist	Ankle, Leg, Waist (optional placement)	Wrist, Upper Arm (Biceps)	Ankle, Waist
Activity Classification	✗	✓	✓	✗	✓	✗
Detailed Gait Analysis	✓	✗	✗	✓	✗	✓
Fall Detection	✗	✓	✓	✓	✗	✓
Suitable for everyday use	✓	✓	✓	✗	✓	✓
Compatible with other ecosystems	✓	✗	✗	✗	✓	✓
AI-powered Suggestion Feature	✗	✓	✓	✗	✓	✓
Post-Accident Gait Detection and Analysis Feature	✗	✗	✗	✗	✗	✓

Figure 2.10: The table that compare existing product with our product

1. **RunScribe Plus (foot-mounted)**
    - **Key Features:**
        - Captures detailed gait metrics: stride length, cadence, ground contact time, pronation/supination.
        - Measures impact forces during walking/running.
        - A foot-mounted IMU sensor ensures high accuracy.
        - Real-time data synchronization with mobile app.
        - **Unique**: Mounted on the foot, providing more accurate gait metrics compared to wrist-worn devices.
    - **Limitations:**
        - Does not include automatic fall detection.

2. **Apple Watch Series 10 (wrist-worn)**
    - **Key Features:**
        - Integrated accelerometer and gyroscope.
        - Fall Detection**:** Sends emergency alerts upon detecting a fall.
        - Comprehensive activity tracking: walking, running, cycling, and other workouts.
        - Health monitoring: heart rate, SpO2, ECG.
        - **Unique:** Seamless integration with the Apple ecosystem; automatic emergency alerts.

- **Limitations**
    - Wrist-based measurement offers lower accuracy for gait metrics compared to foot-mounted devices.

3. **Samsung Galaxy Watch 8 (wrist-worn)**
    - **Key Features:**
        - Built-in accelerometer and gyroscope.
        - Fall Detection + Emergency SOS.
        - Tracks multiple activities: walking, running, swimming, and more.
        - Health monitoring: heart rate, SpO2, ECG.
        - Samsung Health integration: daily activity insights, recovery tracking, and fitness trends.
        - **Unique:** AI-powered coaching and advanced health analytics within the Samsung ecosystem.
    - **Limitations**
        - Wrist placement results in limited gait metric accuracy.

4. **Shimmer3 IMU Unit (research-grade, multi-placement)**
    - **Key Features**
        - High-precision IMU sensors: accelerometer, gyroscope, magnetometer.
        - Flexible placement: foot, leg, waist, back.
        - Raw data capture for biomechanics and rehabilitation research.
        - **Unique:** Provides the highest accuracy for gait and movement pattern analysis, widely used in clinical and research settings.
    - **Limitations**
        - No fall detection capability.
        - Requires manual data processing and technical expertise.
        - Primarily designed for research rather than everyday use.

5. **Whoop 5.0 MG LIFE (wrist/arm-worn)**
    - **Key Features**
        - Wearable for wrist or biceps placement.
        - Tracks heart rate, HRV, sleep, recovery, and strain.
        - AI-driven coaching and performance suggestions.

    - Integrates with Strava, Apple Health, and Google Fit.
    - Designed for continuous 24/7 use without a display.
    - **Unique:** Focuses on recovery and performance optimization with minimal lifestyle interference.
- **Limitations**
    - No gait analysis or fall detection features.
    - Does not provide raw sensor data for research.
    - Requires ongoing subscription fees.

As shown above (Figure 2.10.), while existing solutions offer valuable features such as general gait tracking, heart rate monitoring, and fall detection, none provide a dedicated post-fall gait rehabilitation detection and analysis system. Moreover, most rely on standardized benchmarks that may not reflect an individual's unique walking patterns.

Our product addresses these gaps by introducing personal baseline analysis, accurate rehabilitation tracking, AI-powered suggestions, and a user-friendly design at a significantly lower cost, making it suitable for both everyday users and clinical support.

Unlike some commercial devices, our system does not incorporate general activity classification (e.g., sitting, walking, running) because the primary focus is on post-fall gait rehabilitation and personalized gait analysis where accurate gait pattern monitoring and recovery tracking are more critical than broad activity detection.

2.2 Preliminary Design Based on Backgrounds

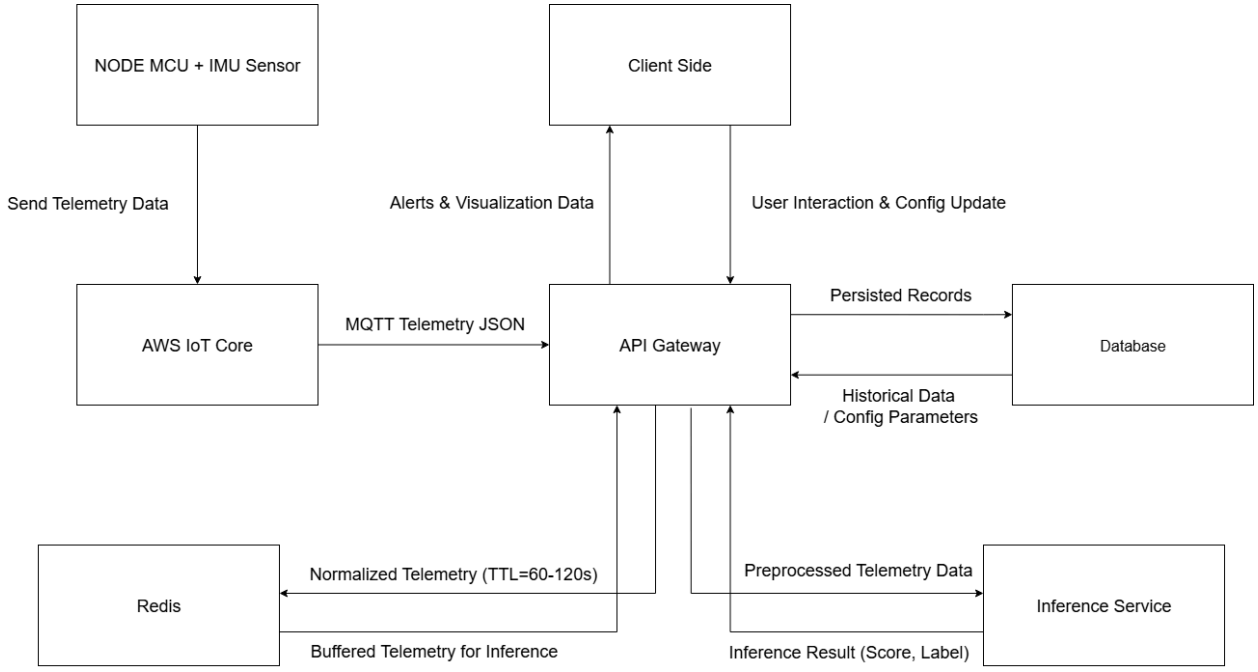


Figure 2.13: Preliminary Data Flow Diagram of the Proposed System

The preliminary design of the proposed system is illustrated through the Data Flow Diagram (DFD), which demonstrates how telemetry data from wearable devices is collected, processed, and utilized. The design begins with a NODE MCU integrated with an IMU sensor, responsible for capturing motion data and transmitting it securely to AWS IoT Core. This cloud-based platform acts as the communication gateway, managing telemetry exchange via the MQTT protocol [29].

From AWS IoT Core, the data is forwarded to the API Gateway, which functions as the central hub of the system. Here, data is distributed to multiple components:

- Redis temporarily buffers normalized telemetry with a time-to-live (TTL) of 60–120 seconds, ensuring efficient handling of high-frequency streams before inference [28].

- Inference Service receives preprocessed telemetry data to generate predictions, producing results in the form of scores or labels that are then relayed back to the API Gateway.
- Database persists long-term records, including telemetry, extracted features, inference outputs, and alerts, while also storing historical data and configuration parameters.
- Client Side applications provide user interaction, system configuration, and real-time visualization of alerts, enabling users and caregivers to make timely decisions.

This preliminary design highlights a clear separation of responsibilities: AWS IoT Core for device connectivity, Redis for short-term buffering, the Inference Service for intelligent analysis, and the Database for reliable storage. The Client Side serves as the human interface, ensuring that insights derived from raw sensor data are delivered in a meaningful and actionable way. By structuring the data flow in this manner, the system balances scalability, real-time responsiveness, and long-term reliability, laying a strong foundation for further development.

References:

1. [1] World Health Organization, “Ageing and health,” Oct. 1, 2024. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/ageing-and-health
2. [2] Hausdorff, J. M., Rios, D. A., & Edelberg, H. K. (2001). Gait variability and fall risk in community-living older adults: a 1-year prospective study. Archives of physical medicine and rehabilitation, 82(8), 1050–1056. Available at: https://www.sciencedirect.com/science/article/abs/pii/S0003999301632155 [Online; accessed 06-September-2025]
3. [3] Paterson K, Hill K, Lythgo N. Stride dynamics, gait variability and prospective falls risk in active community dwelling older women. Gait Posture. 2011 Feb;33(2):251-5. doi: 10.1016/j.gaitpost.2010.11.014. Epub 2010 Dec 16. PMID: 21167715. [Online; accessed 06-September-2025]
4. [4] Verghese, J. et al. (2009). Walking and the risk of falling in older adults. Neurology. Available : https://link.springer.com/article/10.1007/s00415-009-5332-y [Online; accessed 13-September-2025]
5. [5] Rössler, R., Wagner, J., Knaier, R., Rommers, N., Kressig, R. W., Schmidt-Trucksäss, A., & Hinrichs, T. (2024). Spatiotemporal gait characteristics across the adult lifespan: Reference values from a healthy population - Analysis of the COmPLETE cohort study. Gait & posture, 109, 101–108. Available : https://doi.org/10.1016/j.gaitpost.2024.01.005
6. [6] Zijlstra, W., & Hof, A. L. (2003). Assessment of spatio-temporal gait parameters from trunk accelerations during human walking. Gait & posture, 18(2), 1–10. Available : https://doi.org/10.1016/s0966-6362(02)00190-x [Online; accessed 13-September-2025]
7. [7] Casado-Hernández, I., Becerro-de-Bengoa-Vallejo, R., Losa-Iglesias, M., Gómez-Salgado, J., López-López, D., & Bayod, J. (2023). Variability of the lower limb symmetry index associated with the gait parameters in the overweight adult population with flatfoot: a case-control study. Frontiers in bioengineering and biotechnology, 11, 1189309. Available : https://doi.org/10.3389/fbioe.2023.1189309 [Online; accessed 13-September-2025]

8. [8]Iosa M, et al. “Gait phase partitioning in healthy and pathological walking: comparison between accelerometer and gyroscope signals.” Journal of NeuroEngineering and Rehabilitation, 2017;14:47. Available : https://jneuroengrehab.biomedcentral.com/articles/10.1186/s12984-017-0226-1 [Online; accessed 13-September-2025]
9. [9]Bohannon RW. “Comfortable and maximum walking speed of adults aged 20–79 years: reference values and determinants.” Age and Ageing, 1997. Available : https://www.scribd.com/document/496336140/26-1-15 [Online; accessed 13-September-2025]
10. [10]Perry J, Burnfield JM. Gait Analysis: Normal and Pathological Function. 2nd ed. SLACK Incorporated, 2010. Available : https://www.scribd.com/document/608717037/Perry-Burnfield-2010-Gait-Analysis-Normal-and-Pathological-Function [Online; accessed 13-September-2025]
11. [11]Siebers HL, Alrawashdeh W, Betsch M, Migliorini F, Hildebrand F, Eschweiler J. Comparison of different symmetry indices for the quantification of dynamic joint angles. BMC Sports Science, Medicine and Rehabilitation. 2021;13:130. Available : https://doi.org/10.1186/s13102-021-00355-4, [Online; accessed 13-September-2025]
12. [12] Tufte, E. R. The Visual Display of Quantitative Information. 2nd ed. Cheshire, CT: Graphics Press, 2001. Available : https://kyl.neocities.org/books/%5BTEC%20TUF%5D%20the%20visual%20display%20of%20quantitative%20information.pdf [Online; accessed 16-September-2025]
13. [13] Shneiderman, B. The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations. Proceedings 1996 IEEE Symposium on Visual Languages, 1996, pp. 336–343. doi: 10.1109/VL.1996.545307. Available : https://ieeexplore.ieee.org/document/545307 [Online; accessed 16-September-2025]
14. [14] Fogg, B. J. Persuasive Technology: Using Computers to Change What We Think and Do. Morgan Kaufmann, 2003. Available : Persuasive Technologymemoof.mehttps://memoof.me › download › pdf [Online; accessed 16-September-2025]

15. [15] Schork, N. J. Personalized medicine: Time for one-person trials. Nature. 2015 Apr 29;520(7549):609–611. doi: 10.1038/520609a. PMID: 25925459. Available : https://forward.recentiprogressi.it/wp-content/uploads/2015/12/Schork_Personalized-medicine_Time-for-one-person-trials_Nature.pdf [Online; accessed 16-September-2025]
16. [16] ISO 9241-210:2019. Ergonomics of Human-System Interaction – Part 210: Human-Centred Design for Interactive Systems. International Organization for Standardization, 2019. Available : https://cdn.standards.iteh.ai/samples/77520/8cac787a9e1549e1a7ffa0171dfa33e0/ISO-9241-210-2019.pdf [Online; accessed 16-September-2025]
17. [17] Pinto, A., Figueiredo, J., Rodrigues, J., & Sousa, I. (2023). Human Digital Twin-based interactive dashboards for informal caregivers in stroke rehabilitation. Proceedings of the 16th International Conference on PErvasive Technologies Related to Assistive Environments (PETRA ’23), 5–7 July 2023, Corfu, Greece. ACM. doi:10.1145/3594806.3594824. Available : https://dl.acm.org/doi/fullHtml/10.1145/3594806.3594824 [Online; accessed 16-September-2025]
18. [18]Zijlstra & Hof, 2003 – Assessment of spatio-temporal gait parameters from trunk accelerations during human walking, Gait & Posture, 18(2), 1–10. Available : https://pure.rug.nl/ws/files/6673749/Zijlstra_2003_Gait_Posture.pdf [Online; accessed 17-September-2025]
19. [19]Kim et al., 2024 – Determination of Spatiotemporal Gait Parameters Using a Smartphone’s IMU in the Pocket: Threshold-Based and Deep Learning Approaches, Sensors, 25(14), 4395. Available : https://www.mdpi.com/1424-8220/25/14/4395 [Online; accessed 17-September-2025]
20. [20]Springer et al., 2023 – A robust walking detection algorithm using a single foot-worn inertial sensor: validation in real-life settings, Medical & Biological Engineering & Computing, 61, 2683–2695. Available : https://link.springer.com/article/10.1007/s11517-023-02826-x [Online; accessed 17-September-2025]

21. [21] Lee, M.-C., Lin, J.-C., & Katsikas, S., “GAD: A Real-time Gait Anomaly Detection System with Online Adaptive Learning,” arXiv, arXiv:2405.09561, 2024. Available: https://arxiv.org/abs/2405.09561 [Online; accessed 18-September-2025]
22. [22] OASIS Standard, *MQTT Version 5.0*, Mar. 2019. [Online]. Available: [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
23. [23] R. C. Martin, “The Clean Architecture,” *The Clean Code Blog*, Aug. 13, 2012. Available: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html [Online]
24. [24] Meta Platforms, Inc., “React Native – a framework for building native apps using React,” *React Native*, 2025. [Online]. Available: https://reactnative.dev/
25. [25] Jinzhu Zhang, “GORM – The fantastic ORM library for Golang, aims to be developer friendly,” *GORM*, 2025. [Online]. Available: https://gorm.io/index.html
26. [26] Fiber Authors, “Fiber – Express inspired web framework written in Go,” *Fiber Web Framework*, 2025. [Online]. Available:https://gofiber.io/
27. [27] PostgreSQL Global Development Group, “About PostgreSQL,” *PostgreSQL*, 2025. [Online]. Available: https://www.postgresql.org/about/
28. [28] Redis Ltd., “Redis: The real-time data platform,” *Redis*, 2025. [Online]. Available: https://redis.io/
29. [29] Amazon Web Services, Inc., “AWS IoT Core,” *Amazon Web Services*, 2025. [Online]. Available: https://aws.amazon.com/iot-core/
30. [30] Anne Fouilloux. *Introduction to IoT – ESP8266*. Available: https://annefou.github.io/IoT_introduction/02-ESP8266/index.html [Online; accessed 17-September-2025]
31. [31] Adafruit. (n.d.). *Adafruit 9-DOF Orientation IMU Fusion Breakout - BNO085*. Available at: https://learn.adafruit.com/adafruit-9-dof-orientation-imu-fusion-breakout-bno085/overview [Accessed 19 September 2025].
32. [32] Hsu, W.-C., Sugiarto, T., Lin, Y.-J., Yang, F.-C., Lin, Z.-Y., Sun, C.-T., Hsu, C.-L., & Chou, K.-N. (2018) — Multiple-Wearable-Sensor-Based Gait Classification and Analysis in Patients with Neurological Disorders. *Sensors*, 18(10): 3397. Available:

https://pmc.ncbi.nlm.nih.gov/articles/PMC6210399/ [Online; accessed 19-September-2025]

33. [33] Storm, F. A., Cesareo, A., Reni, G., & Biffi, E. (2020) — *Wearable Inertial Sensors to Assess Gait during the 6-Minute Walk Test: A Systematic Review*. *Sensors (Basel)*, 20(9): 2660. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC7249076/ [Online; accessed 19-September-2025]
34. [34] C. R. Harris, K. J. Millman, S. J. van der Walt, et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020. doi:10.1038/s41586-020-2649-2. Available : https://www.nature.com/articles/s41586-020-2649-2 [Online; accessed 18-September-2025]
35. [35] W. McKinney, "Data structures for statistical computing in Python," in Proceedings of the 9th Python in Science Conference (SciPy 2010), pp. 51–56, 2010. Available : https://proceedings.scipy.org/articles/Majora-92bf1922-00a [Online; accessed 18-September-2025]
36. [36] P. Virtanen, R. Gommers, T. E. Oliphant, et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, pp. 261–272, 2020. doi:10.1038/s41592-019-0686-2. Available : https://www.nature.com/articles/s41592-019-0686-2 [Online; accessed 18-September-2025]
37. [37] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. Available : Scikit-learn: Machine Learning in PythonJournal of Machine Learning Researchhttps://jmlr.org › papers › volume12 [Online; accessed 18-September-2025]
38. [38] M. Abadi, A. Agarwal, P. Barham, et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org [Online; accessed 18-September-2025]
39. [39] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007. doi:10.1109/MCSE.2007.55. Available : https://ieeexplore.ieee.org/document/4160265 [Online; accessed 18-September-2025]

40. [40] M. Waskom, “Seaborn: Statistical data visualization,” Journal of Open Source Software, vol. 6, no. 60, p. 3021, 2021. doi:10.21105/joss.03021. Available : https://joss.theoj.org/papers/10.21105/joss.03021 [Online; accessed 18-September-2025]
41. [41] Google Research, “Google Colaboratory,” 2025. [Online]. Available: https://colab.research.google.com [Online; accessed 18-September-2025]
42. [42] S. Ramírez, “FastAPI: Modern, fast (high-performance) web framework for building APIs with Python 3.7+,” 2018. [Online]. Available: https://fastapi.tiangolo.com [Online; accessed 18-September-2025]